

```
}e1; //declaring e1 variable for structure  
  
int main( )  
{  
    //store first employee information
```

```

e1.id=101;

strcpy(e1.name, "Sonoo Jaiswal");//copying string into char array

//printing first employee information

printf( "employee 1 id : %d\n", e1.id);

printf( "employee 1 name : %s\n", e1.name);

return 0;

}

```

Output:

employee 1 id : 101

employee 1 name : Sonoo Jaiswal

Difference Between Array and Structure

1	Array is collection of homogeneous data.	Structure is the collection of heterogeneous data.
2	Array data are access using index.	Structure elements are access using . operator.
3	Array allocates static memory.	Structures allocate dynamic memory.
4	Array element access takes less time than structures.	Structure elements takes more time than Array.

Nested Structures

structure can have another structure as a member. There are two ways to define nested structure in c language:

1. By separate structure
2. By Embedded structure

1) Separate structure

We can create 2 structures, but dependent structure should be used inside the main structure as a

member. Let's see the code of nested structure.

```
struct Date
```

```
{
```

```
    int dd;
```

```
    int mm;
```

```
    int yyyy;
```

```
};
```

```
struct Employee
```

```
{
```

```
    int id;
```

```
    char name[20];
```

```
    struct Date doj;
```

```
}emp1;
```

2) Embedded structure

```
struct Employee
```

```
{
```

```
    int id;
```

```
    char name[20];
```

```
    struct Date
```

```
    {
```

```
        int dd;
```

```
        int mm;
```

```
        int yyyy;
```

```
    }doj;
```

```
}emp1;
```

Accessing Nested Structure

We can access the member of nested structure by `Outer_Structure.Nested_Structure.member` as given below:

`e1.doj.dd`

`e1.doj.mm`

`e1.doj.yyyy`

Arrays of Structures

Array of structures to store much information of different data types. Each element of the array representing a **structure** variable. The array of structures is also known as collection of structures.

Ex : if you want to handle more records within one structure, we need not specify the number of structure variable. Simply we can use array of structure variable to store them in one structure variable.

Example : `struct employee emp[5];`

Example of structure with array that stores information of 5 students and prints it.

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

struct student{

    int rollno;

    char name[10];

};

void main(){

    int i;

    struct student st[5];

    clrscr();
```

```
printf("Enter Records of 5 students");  
  
for(i=0;i<5;i++){  
    printf("\nEnter Rollno:");  
    scanf("%d",&st[i].rollno);  
    printf("\nEnter Name:");  
    scanf("%s",&st[i].name);  
}  
  
printf("\nStudent Information List:");  
  
for(i=0;i<5;i++){  
    printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);  
}  
  
getch();  
}
```

Output:

Enter Records of 5 students

Enter Rollno:1

Enter Name:Sonoo

Enter Rollno:2

Enter Name:Ratan

Enter Rollno:3

Enter Name:Vimal

Enter Rollno:4

Enter Name:James

Enter Rollno:5

Enter Name:Sarfraz

Student Information List:

Rollno:1, Name:Sonoo

Rollno:2, Name:Ratan

Rollno:3, Name:Vimal

Rollno:4, Name:James

Rollno:5, Name:Sarfraz

Structures and Functions

A structure can be passed as a function argument just like any other variable. This raises a few practical issues.

PASSING STRUCTURE TO FUNCTION IN C:

It can be done in below 3 ways.

1. Passing structure to a function by value
2. Passing structure to a function by address(reference)
3. No need to pass a structure – Declare structure variable as global

.

The general format of sending a copy of a structure to the called function is:

Function_name(structure_variable_name);

The called function takes the following form:

```
data_type function_name(struct tag_name var)  
{
```

```

        -----
        -----

    return(exp);

}

```

PASSING STRUCTURE TO FUNCTION IN C BY VALUE:

```

#include <stdio.h>

#include <string.h>

struct student
{
    int id;
    char name[20];
    float percentage;
};

void func(struct student record);

int main()
{
    struct student record;
    record.id=1;
    strcpy(record.name, "Raju");
    record.percentage = 86.5;
    func(record);
    return 0;
}

void func(struct student record)

```



```
{  
    printf(" Id is: %d \n", record.id);  
    printf(" Name is: %s \n", record.name);  
    printf(" Percentage is: %f \n", record.percentage);  
}
```

Output

Id is: 1

Name is: Raju

Percentage is: 86.500000

PASSING STRUCTURE TO FUNCTION IN C BY ADDRESS:

```
#include <stdio.h>  
#include <string.h>  
struct student  
{  
    int id;  
    char name[20];  
    float percentage;  
};  
void func(struct student *record);  
int main()  
{  
    struct student record;  
    record.id=1;  
    strcpy(record.name, "Raju");  
    record.percentage = 86.5;
```

```

        func(&record);

    return 0;
}

void func(struct student *record)
{
    printf(" Id is: %d \n", record->id);
    printf(" Name is: %s \n", record->name);
    printf(" Percentage is: %f \n", record->percentage);
}

```

EXAMPLE PROGRAM TO DECLARE A STRUCTURE VARIABLE AS GLOBAL IN C:

```

#include <stdio.h>
#include <string.h>

struct student
{
    int id;
    char name[20];
    float percentage;
};

struct student record; // Global declaration of structure

void structure_demo();

int main()
{
    record.id=1;
}

```

```

        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        structure_demo();
        return 0;
    }

void structure_demo()
{
    printf(" Id is: %d \n", record.id);
    printf(" Name is: %s \n", record.name);
    printf(" Percentage is: %f \n", record.percentage);
}

```

Passing a copy of entire structure to a function

```

struct std
{
    int no;
    float avg;
};

struct std a;

void fun(struct std p);

void main()

```

```

{
clrscr();
a.no=12;
a.avg=13.76;
fun(a);
getch();
}

void fun(struct std p)
{
printf("number is%d\n",p.no);
printf("average is%f\n",p.avg);
}

```

Passing Structures through Pointers

Example

```

#include <stdio.h>

#include <string.h>

struct student
{
    int id;

    char name[30];

    float percentage;

```

```

};

int main()
{
    int i;

    struct student record1 = {1, "Raju", 90.5};

    struct student *ptr;

    ptr = &record1;

    printf("Records of STUDENT1: \n");

    printf(" Id is: %d \n", ptr->id);

    printf(" Name is: %s \n", ptr->name);

    printf(" Percentage is: %f \n\n", ptr->percentage);


    return 0;
}

```

OUTPUT:

Records	of	STUDENT1:
Id	is:	1
Name is: Raju		

Percentage is: 90.500000

Self-referential Structures

A structure consists of at least a pointer member pointing to the same structure is known as a self-referential structure. A self referential structure is used to create data structures like linked lists, stacks, etc. Following is an example of this kind of structure:

A self-referential structure is one of the data structures which refer to the pointer to (points) to another structure of the same type. For example, a linked list is supposed to be a self-referential data structure. The next node of a node is being pointed, which is of the same struct type. For example,

Syntax : struct tag_name

```
{  
    type member1;  
    type member2;  
    :  
    :  
    typeN memberN;  
    struct tag_name *name;  
}
```

Where *name refers to the name of a pointer variable.

Ex:

```
struct emp  
{  
    int code;  
    struct emp *name;  
}
```

Unions

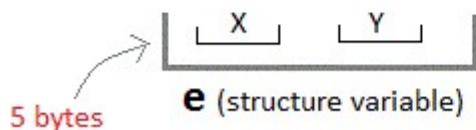
A union is a special data type available in C that allows to store different data types in the same memory location.

Unions are conceptually similar to **structures**. The syntax of **union** is also similar to that of structure. The only difference is in terms of storage. In **structure** each member has its own storage location, whereas all members of **union** use a single shared memory location which is equal to the size of its largest data member.

We can access only one member of union at a time. We can't access all member values at the same time in union. But, structure can access all member values at the same time. This is because, Union allocates one common storage space for all its members. Where **as** Structure allocates storage space for all its members separately.

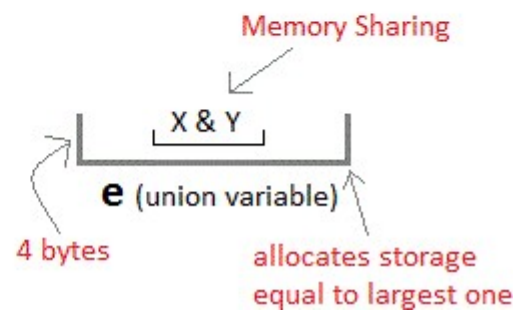
Structure

```
struct Emp
{
    char X;    // size 1 byte
    float Y;   // size 4 byte
} e;
```



Unions

```
union Emp
{
    char X;
    float Y;
} e;
```



Structure

```
struct Employee{  
char x; // size 1 byte  
int y; //size 2 byte  
float z; //size 4 byte  
}e1; //size of e1 = 7 byte
```

size of e1= 1 + 2 + 4 = 7

Union

```
union Employee{  
char x; // size 1 byte  
int y; //size 2 byte  
float z; //size 4 byte  
}e1; //size of e1 = 4 byte
```

size of e1= 4 (maximum size of 1 element)

JavaTpoint.com

syntax

```
union union_name  
{  
    data_type member1;  
    data_type member2;  
    .  
    .  
    data_type memberN;  
};
```

Example

```
union employee  
{  
    int id;  
    char name[50];  
    float salary;  
};
```

Example


```

#include <stdio.h>

#include <string.h>

union employee
{
    int id;

    char name[50];
}e1; //declaring e1 variable for union

int main( )
{
    //store first employee information

    e1.id=101;

    strcpy(e1.name, "Sonoo Jaiswal");//copying string into char array

    //printing first employee information

    printf( "employee 1 id : %d\n", e1.id);

    printf( "employee 1 name : %s\n", e1.name);

    return 0;
}

```

Output:

employee 1 id : 1869508435

employee 1 name : Sonoo Jaiswal

As you can see, id gets garbage value because name has large memory size. So only name will have actual value.

Example

```

#include <stdio.h>

#include <conio.h>

union item

```

C PROGRAMMING


```

{
    int a;

    float b;

    char ch;

};

int main( )

{
    union item it;

    it.a = 12;

    it.b = 20.2;

    it.ch='z';

    clrscr();

    printf("%d\n",it.a);

    printf("%f\n",it.b);

    printf("%c\n",it.ch);

    getch();

    return 0;

}

```

Output

-26426

20.1999

z

As you can see here, the values of **a** and **b** get corrupted and only variable **c** prints the expected result. Because in **union**, the only member whose value is currently stored will have the memory.

Difference between Structure and Union

	Structure	Union
1	For defining structure use struct keyword.	For defining union we use union keyword
2	Structure occupies more memory space than union.	Union occupies less memory space than Structure.
3	In Structure we can access all members of structure at a time.	In union we can access only one member of union at a time.
4	Structure allocates separate storage space for its every members.	Union allocates one common storage space for its all members. Union find which member need more memory than other member, then it allocate that much space

Bit-Fields

Syntax

```
struct {  
    type [member_name] : width ;  
};
```

The following table describes the variable elements of a bit field –

Elements	Description
type	An integer type that determines how a bit-field's value is interpreted. The type may be int, signed int, or unsigned int.
member_name	The name of the bit-field.
width	The number of bits in the bit-field. The width must be less than or equal to the bit width of the specified type.

The variables defined with a predefined width are called **bit fields**. A bit field can hold more than a single bit; for example, if you need a variable to store a value from 0 to 7, then you can define a bit field with a width of 3 bits as follows –

```
struct {  
    unsigned int age : 3;  
} Age;
```

The above structure definition instructs the C compiler that the age variable is going to use only 3 bits to store the value. If you try to use more than 3 bits, then it will not allow you to do so. Let us try the following example –

```
#include <stdio.h>  
  
#include <string.h>  
  
struct {  
    unsigned int age : 3;  
} Age;  
  
int main( ) {  
    Age.age = 4;  
  
    printf( "Sizeof( Age ) : %d\n", sizeof(Age) );  
  
    printf( "Age.age : %d\n", Age.age );  
  
    Age.age = 7;  
  
    printf( "Age.age : %d\n", Age.age );  
  
    Age.age = 8;  
  
    printf( "Age.age : %d\n", Age.age );  
  
    return 0;  
}
```

Output

```
Sizeof( Age ) : 4
```

Age.age : 4

Age.age : 7

Age.age : 0

typedef

The typedef is a keyword that allows the programmer to create a new data type name for an existing data type. So, the purpose of typedef is to redefine the name of an existing variable type.

Syntax

```
typedef datatype alias_name;
```

Example of typedef

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
typedef int Intdata; // Intdata is alias name of int
```

```
void main()
```

```
{
```

```
int a=10;
```

```
Integerdata b=20;
```

```
typedef Intdata Integerdata; // Integerdata is again alias name of Intdata
```

```
Integerdata s;
```

```
clrscr();
```

```
s=a+b;
```

```
printf("\n Sum:= %d",s);
```

```
getch();
```

```
}
```

Output

Sum: 20

Advantages of typedef :

- 1** : Provides a meaningful way of declaring the variable.
- 2** : Increase the readability of the program.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    typedef int digits;
    digits a,b,sum;
    clrscr();
    printf("Enter a and b values:");
    scanf("%d%d",&a,&b);
    sum=a+b;
    printf("The sum is:%d",sum);
    getch();
}
```

```
File Edit Search Run Compile Debug Project Options Window Help
\TC\TYPEDE~1.C
#include <stdio.h>
#include <conio.h>

typedef int myint;
void main()
{
    int a=10;
    myint b=20;
    typedef myint smallint;
    smallint s;
    clrscr();
    s=a+b;
    printf("\n Sum:= %d",s);
    getch();
}
```

alias name myint of int
this is act as integer

alias name smallint of
myint this is act as
integer

alias name s of smallint
this is act as integer

17:9

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Note: By using typedef only we can create the alias name and it is under control of compiler.

Application of typedef

typedef can be used to give a name to user defined data type as well. Lets see its use with structures.

typedef struct

```
{
    type member1;
    type member2;
    type member3;
} type_name ;
```


Here **type_name** represents the structure definition associated with it. Now this **type_name** can be used to declare a variable of this structure type.

```
type_name t1, t2 ;
```

Example of structure definition using typedef

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
typedef struct employee
```

```
{
```

```
    char name[50];
```

```
    int salary;
```

```
} emp ;
```

```
void main( )
```

```
{
```

```
    emp e1;
```

```
    printf("\nEnter Employee record\n");
```

```
    printf("\nEnter Employee name\t");
```

```
    scanf("%s",e1.name);
```

```
    printf("\nEnter Employee salary \t");
```

```
    scanf("%d",&e1.salary);
```

```
    printf("\nEnter student name is %s",e1.name);
```

```
    printf("\nEnter roll is %d",e1.salary);
```

```
    getch();
```

```
}
```

typedef and Pointers

typedef can be used to give an alias name to pointers also. Here we have a case in which use of typedef is beneficial during pointer declaration.

In Pointers * binds to the right and not the left.

```
int* x, y ;
```

By this declaration statement, we are actually declaring **x** as a pointer of type int, whereas **y** will be declared as a plain integer.

```
typedef int* IntPtr ;
```

```
IntPtr x, y, z;
```

But if we use **typedef** like in above example, we can declare any number of pointers in a single statement.

NOTE : If you do not have any prior knowledge of pointers, do study Pointers first.

Enumerations

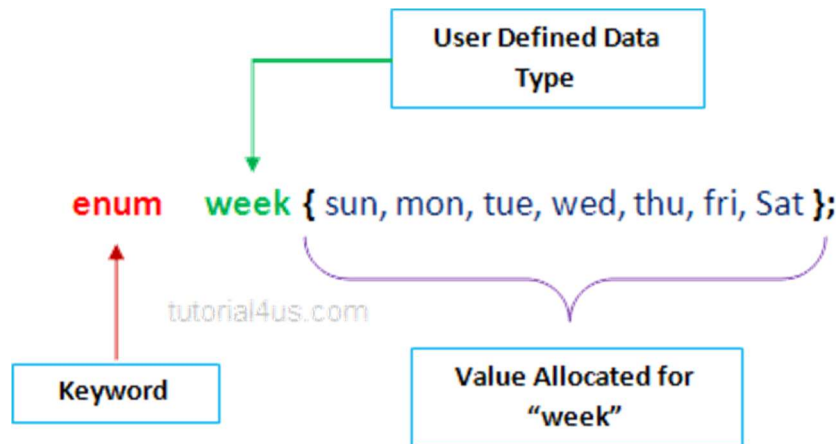
An **enum** is a keyword, it is an user defined data type. All properties of integer are applied on Enumeration data type so size of the enumerator data type is 2 byte. It work like the Integer.

It is used for creating an user defined data type of integer. Using enum we can create sequence of integer constant value.

Syntax

```
enum tagname {value1, value2, value3,... };
```

- In above syntax **enum** is a keyword. It is a user defiend data type.
- In above syntax tagname is our own variable. tagname is any variable name.
- value1, value2, value3,.....are create set of enum values.



It starts with 0 (zero) by default and the value is incremented by 1 for the sequential identifiers in the list. If a constant value is not initialized, then by default the sequence will start from zero and the next generated value should be the previous constant value plus one.

Example of Enumeration in C

```
#include<stdio.h>

#include<conio.h>

enum ABC {x,y,z};

void main()
{
    int a;

    clrscr();

    a=x+y+z;    //0+1+2

    printf("Sum: %d",a);

    getch();
}
```

Output

Sum: 3

Example of Enumeration in C

```
#include<stdio.h>
```

```
#include<conio.h>

enum week {sun, mon, tue, wed, thu, fri, sat};

void main()

{

enum week today;

today=tue;

printf("%d day",today+1);

getch();

}
```

Output

3 day

Example of Enumeration in C

```
#include<stdio.h>

#include<conio.h>

enum week {sun, mon, tue, wed, thu, fri, sat};

void main()

{

for(i=sun; i<=sat; i++)

{

printf("%d ",i);

}

getch();

}
```

Output

In above code replace sun, mon, tue,... with Equivalent numeric value 0, 1, 2,...

UNIT – V

FILE

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all. However, if you have a file containing all the data, you can easily access the contents of the file using few commands in C.
- You can easily move your data from one computer to another without any changes.

File I/O:-

Sometimes it is necessary to store the data in a manner that can be later retrieved and displayed either in a part or in whole. This medium is usually a “file” on the disk. File I/O can be handled by using different functions.

a) Formatted functions:- The file input function `fscanf()` and the file output function `fprintf()` are called formatted file I/O functions.

b) Unformatted functions:- The input functions like `getc()`, `getw()`, and `fread()` are called unformatted file input functions and `putc()`, `putw()`, and `fwrite()` functions are unformatted file output functions. Each and every function is having its own syntax and meaning.

File streams:- Stream is either reading or writing of data. The streams are designed to allow the user to access the files efficiently. A stream is a file or physical device like key board, printer, monitor, etc., The FILE object uses these devices. When a C program is started, the operating system is responsible for opening three streams: standard input stream (**`stdin`**), standard output stream (**`stdout`**), standard error(**`stderr`**). Normally the `stdin` is connected to the keyboard, the `stdout` and `stderr` are connected to the monitor.

Files

File is a collection of bytes that is stored on secondary storage devices like Hard disk.

OR

A **file** represents a sequence of bytes on the disk where a group of related data is stored. File is created for permanent storage of data. It is a ready made structure.

Note:

C PROGRAMMING

All files related function are available in **stdio.h** header file.

Types of Files

When dealing with files, there are two types of files you should know about:

1. Text files
2. Binary files

1. Text files

Text files are the normal .txt files that you can easily create using Notepad or any simple text editors.

When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.

They take minimum effort to maintain, are easily readable, and provide least security and takes bigger storage space.

2. Binary files

Binary files are mostly the .bin files in your computer.

Instead of storing data in plain text, they store it in the binary form (0's and 1's).

They can hold higher amount of data, are not readable easily and provides a better security than text files.

File Operations

In C, you can perform four major operations on the file, either text or binary:

- Naming a file/Creation of new file
- Opening an existing file
- Reading data from file
- Writing data into file
- Closing a file

Steps for processing a file

- Declare a file pointer
- open a file using fopen() function
- Process the file using suitable file functions.
- close the file using fclose() function.

Declaration of a file

When working with files, you need to declare a pointer of type file. This declaration is needed for communication between the file and program.

Syntax

```
FILE *fp;
```

Opening a file - for creation and edit

The fopen() function is used to create a new file or to open an existing file.

General Syntax :

```
fp = fopen("filename","mode")
```

For Example:

```
fopen("E:\\cprogram\\newprogram.txt","w");
```

```
fopen("E:\\cprogram\\oldprogram.bin","rb");
```

Closing a File

The file (both text and binary) should be closed after reading/writing.

Closing a file is performed using library function fclose().

```
fclose(fp); //fp is the file pointer associated with file to be closed.
```

File Opening Modes

Mode	Description
r	opens a text file in read mode
w	opens a text file in write mode
a	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode
a+	opens a text file in read and write mode
rb	opens a binary file in read mode

wb	opens a binary file in write mode
ab	opens a binary file in append mode
rb+	opens a binary file in read and write mode
wb+	opens a binary file in read and write mode
ab+	opens a binary file in read and write mode

Difference between Append and Write Mode

Write (w) mode and Append (a) mode, while opening a file are almost the same. Both are used to write in a file. In both the modes, new file is created if it doesn't exist already.

The only difference they have is, when you open a file in the write mode, the file is reset, resulting in deletion of any data already present in the file. While in append mode this will not happen. Append mode is used to append or add data to the existing data of file(if any). Hence, when you open a file in Append(a) mode, the cursor is positioned at the end of the present data in the file.

Formatted File I/O Functions

Syntax of fprintf is

```
fprintf(fp, "control string", list);
```

Example: `fprintf(fp1, "%s %d", name, age);`

Syntax of fscanf is,

```
fscanf(fp, "control string", list);
```

Example: `fscanf(fp, "%s %d", name, & age);`

Note:

- fscanf is used to read list of items from a file
- fprintf is used to write a list of items to a file.

Note:

EOF – End of file (when EOF encountered the reading / writing should be terminated)

Example:

```
#include <stdio.h>

main(){
    FILE *fp;

    fp = fopen("file.txt", "w");//opening file
    fprintf(fp, "Hello file by fprintf...\n");//writing data into file
    fclose(fp);//closing file
}
```

Example 1: Write to a text file using fprintf()

```
#include <stdio.h>

int main()
{
    int num;
    FILE *fptr;
    fptr = fopen("C:\\program.txt","w");
    if(fptr == NULL)
    {
        printf("Error!");
        exit(1);
    }
    printf("Enter num: ");
    scanf("%d",&num);
    fprintf(fptr,"%d",num);
    fclose(fptr);
    return 0;
}
```

Example 2: Read from a text file using fscanf()

```
#include <stdio.h>

int main()
{
    int num;
    FILE *fptr;
    if ((fptr = fopen("C:\\program.txt", "r")) == NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }

    fscanf(fptr, "%d", &num);
    printf("Value of n=%d", num);
    fclose(fptr);
    return 0;
}
```

Input/Output Operation on files

To perform Input/Output Operation on files we need below functions.

S.No	Function	Operation	Syntax
1	getc()	Read a character from a file	getc(fp)
2	putc()	Write a character in file	putc(c, fp)

3	fprintf()	To write set of data in file	fprintf(fp, "control string", list)
4	fscanf()	To read set of data from file.	fscanf(fp, "control string", list)
5	getw()	To read an integer from a file.	getw(fp)
6	putw()	To write an integer in file.	putw(integer, fp)

Unformatted File I/O Functions

fputc() function

The fputc() function is used to write a single character into file.

putc ():-Putting a character in to the file. It works with only character data type. One character at a time can write into a file.

Ex: char ch = 'a';

```
putc (ch, fp);
```

Example:

```
#include <stdio.h>

main(){
    FILE *fp;
    fp = fopen("file1.txt", "w");//opening file
    fputc('a',fp);//writing single character into file
    fclose(fp);//closing file
}
```

file1.txt

a

fgetc() function

The fgetc() function returns/read a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

getc (): getting a character from the file, or reading the file information character by character at a time, upto the end of the file by using this function.

Ex: char ch;
 ch = getc (fp);

Example:

```
#include<stdio.h>

#include<conio.h>

void main(){

FILE *fp;

char c;

clrscr();

fp=fopen("myfile.txt","r");

while((c=fgetc(fp))!=EOF){

printf("%c",c);

}

fclose(fp);

getch();

}
```

myfile.txt

this is simple text message

fputs()

The fputs() function writes a line of characters into file

Example:

```
#include<stdio.h>
```

```
#include<conio.h>

void main(){

FILE *fp;

clrscr();

fp=fopen("myfile2.txt","w");

fputs("hello c programming",fp);

fclose(fp);

getch();

}
```

myfile2.txt

hello c programming

fgets()

The fgets() function reads a line of characters from file.

Example:

```
#include<stdio.h>

#include<conio.h>

void main(){

FILE *fp;

char text[300];

clrscr();

fp=fopen("myfile2.txt","r");

printf("%s",fgets(text,200,fp));

fclose(fp);
```

```
    getch();  
}
```

Output:

hello c programming

The getw and putw functions:

These are integer oriented functions. These are similar to above functions and are used to read and write integer values. These are useful when we deal with only integer data. The general format is

putw (): putting or writing_of an integer value to a file.

```
putw (integer , fp);
```

Ex: int x = 5;
 putw(x,fp);

getw (): getting or reading integer value from a file.

Ex: int x;
 x = getw (fp);

File Positioning Functions**fseek()**

The fseek() function is used to set the file pointer to the specified offset. It is used to write data into file at desired location.

syntax:

```
fseek(FILE * stream, long int offset, int whence)
```

The first parameter stream is the pointer to the file. The second parameter is the position of the record to be found, and the third parameter specifies the location where the offset starts.

Different Whence in fseek	
Whence	Meaning
SEEK_SET	Starts the offset from the beginning of the file.
SEEK_END	Starts the offset from the end of the file.
SEEK_CUR	Starts the offset from the current location of the cursor in the file.

or

fseek(file pointer, offset, position);

- *file pointer* is a pointer to the concerned file.
- *Offset* is a number or variable of type long, it specifies the number of positions (bytes) to be moved from the location specified. If offset is positive number, then moving forward or negative meaning move backwards.
- *Position* is a n integer number and it specifies from which position the file pointer to be moved. Position can take one of the following three values.

0	beginning of file
1	current position
2	end of file

Eg: fseek (fp, 0L,0); - go to the beginning of the file. (Similar to rewind).
 fseek (fp, 0L,1); - Stay at current position (Rarely used)
 fseek (fp, 0L,2); -go to the end of the file, past the last character of the file.

Example:

```
#include <stdio.h>

void main(){
```



```
FILE *fp;

fp = fopen("myfile.txt","w+");

fputs("This is javatpoint", fp);

fseek( fp, 7, SEEK_SET );

fputs("sonoo jaiswal", fp);

fclose(fp);

}
```

myfile.txt

This is sonoo jaiswal

rewind()

This function places the file pointer to the beginning of the file, irrespective of where it is present right now. It takes file pointer as an argument.

Syntax:

```
rewind( fp);
```

Example:

File: file.txt

this is a simple text

Example:

```
#include<stdio.h>

#include<conio.h>

void main(){
```

```

FILE *fp;

char c;

clrscr();

fp=fopen("file.txt","r");

while((c=fgetc(fp))!=EOF){

printf("%c",c);

}

rewind(fp);//moves the file pointer at beginning of the file

while((c=fgetc(fp))!=EOF){

printf("%c",c);

}

fclose(fp);

getch();

}

```

Output:

this is a simple textthis is a simple text

As you can see, rewind() function moves the file pointer at beginning of the file that is why "this is simple text" is printed 2 times. If you don't call rewind() function, "this is simple text" will be printed only once.

ftell()

The ftell() function returns the current file position of the specified stream. We can use ftell() function to get the total size of a file after moving file pointer at the end of file. We can use SEEK_END constant to move the file pointer at the end of file.

syntax:

```
n = ftell(fp);
```

n would give the relative offset(in bytes).

Example:

```
#include <stdio.h>

#include <conio.h>

void main (){

    FILE *fp;

    int length;

    clrscr();

    fp = fopen("file.txt", "r");

    fseek(fp, 0, SEEK_END);

    length = ftell(fp);

    fclose(fp);

    printf("Size of file: %d bytes", length);

    getch();

}
```

Output:

Size of file: 21 bytes

INBUILT FUNCTIONS FOR FILE HANDLING IN C LANGUAGE:

File handling functions	Description
<u>fopen ()</u>	fopen () function creates a new file or opens an existing file.
<u>fclose ()</u>	fclose () function closes an opened file.
<u>getw ()</u>	getw () function reads an integer from file.

<u>putw ()</u>	putw () functions writes an integer to file.
<u>fgetc ()</u>	fgetc () function reads a character from file.
<u>fputc ()</u>	fputc () functions write a character to file.
<u>gets ()</u>	gets () function reads line from keyboard.
<u>puts ()</u>	puts () function writes line to o/p screen.
<u>fgets ()</u>	fgets () function reads string from a file, one line at a time.
<u>fputs ()</u>	fputs () function writes string to a file.
<u>feof ()</u>	feof () function finds end of file.
<u>fgetchar ()</u>	fgetchar () function reads a character from keyboard.
<u>fprintf ()</u>	fprintf () function writes formatted data to a file.
<u>fscanf ()</u>	fscanf () function reads formatted data from a file.
<u>fputchar ()</u>	fputchar () function writes a character onto the output screen from keyboard input.
<u>fseek ()</u>	fseek () function moves file pointer position to given location.
<u>SEEK_SET</u>	SEEK_SET moves file pointer position to the beginning of the file.

<u>SEEK_CUR</u>	SEEK_CUR moves file pointer position to given location.
<u>SEEK_END</u>	SEEK_END moves file pointer position to the end of file.
<u>ftell ()</u>	ftell () function gives current position of file pointer.
<u>rewind ()</u>	rewind () function moves file pointer position to the beginning of the file.
<u>getc ()</u>	getc () function reads character from file.
<u>getch ()</u>	getch () function reads character from keyboard.
<u>getche ()</u>	getche () function reads character from keyboard and echoes to o/p screen.
<u>getchar ()</u>	getchar () function reads character from keyboard.
<u>putc ()</u>	putc () function writes a character to file.
<u>putchar ()</u>	putchar () function writes a character to screen.
<u>printf ()</u>	printf () function writes formatted data to screen.
<u>sprintf ()</u>	sprintf () function writes formatted output to string.
<u>scanf ()</u>	scanf () function reads formatted data from

	keyboard.
<u>sscanf ()</u>	sscanf () function Reads formatted input from a string.
<u>remove ()</u>	remove () function deletes a file.
<u>fflush ()</u>	fflush () function flushes a file.

